Mahidol University
Wisdom of the Land

Mahidol University

ITDS241 Web Technologies and Applications



ECMAScript Version

Instructor: Asst. Prof. Wudhichart Sawangphol, Ph.D., SFHEA
Email: wudhichart.saw@mahidol.ac.th

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Class Objectives

- Students will be able
  - To explain the basic concept of each ECMAScript version
  - To identify the features in each ECMAScript version

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Contents

- What is ECMAScript ?
- ECMAScript versions

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ECMAScript

- ECMAScript (ES) is a general-purpose programming language
- It is standardized by Ecma International
- JavaScript is the most popular and widely used implementation of ES.
- In this lecture, the ES version of JS will be described.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ECMAScript versions

Edition	Official name	Date published
ES6	ES2015	June 2015
ES5.1	ES5.1	June 2011
ES5	ES5	December 2009
ES4	ES4	Abandoned
ES3	ES3	December 1999
ES2	ES2	June 1998
ES1	ES1	June 1997

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ECMAScript versions

Edition	Official name	Date published
ES13	ES2022	June 2022
ES12	ES2021	June 2021
ES11	ES2020	June 2020
ES10	ES2019	June 2019
ES9	ES2018	June 2018
ES8	ES2017	June 2017
ES7	ES2016	June 2016

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ECMAScript versions

Edition	Official name	Date published
ES17	ES2026	June 2026
ES16	ES2025	June 2025
ES15	ES2024	June 2024
ES14	ES2023	June 2023

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.





ES6 (ES2015)

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES6 (ES2015)

- Arrow Functions
- let and const
- Classes
- Promises
- Template Literals
- Default parameters
- The spread operator
- Destructuring assignments
- For-of loop
- Map and Set
- New String and Object methods
- Function Rest Parameter

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES6 (ES2015): Promises

- The **Promise** object represents the eventual completion (or failure) of an asynchronous operation and its resulting value.
- “Producing code” – does somethings and take time, e.g., Loading data from database
- “Consuming code” – get the result from the “producing code” when it is ready
- The **Promise** object is the link between the “producing code” and the “consuming code”

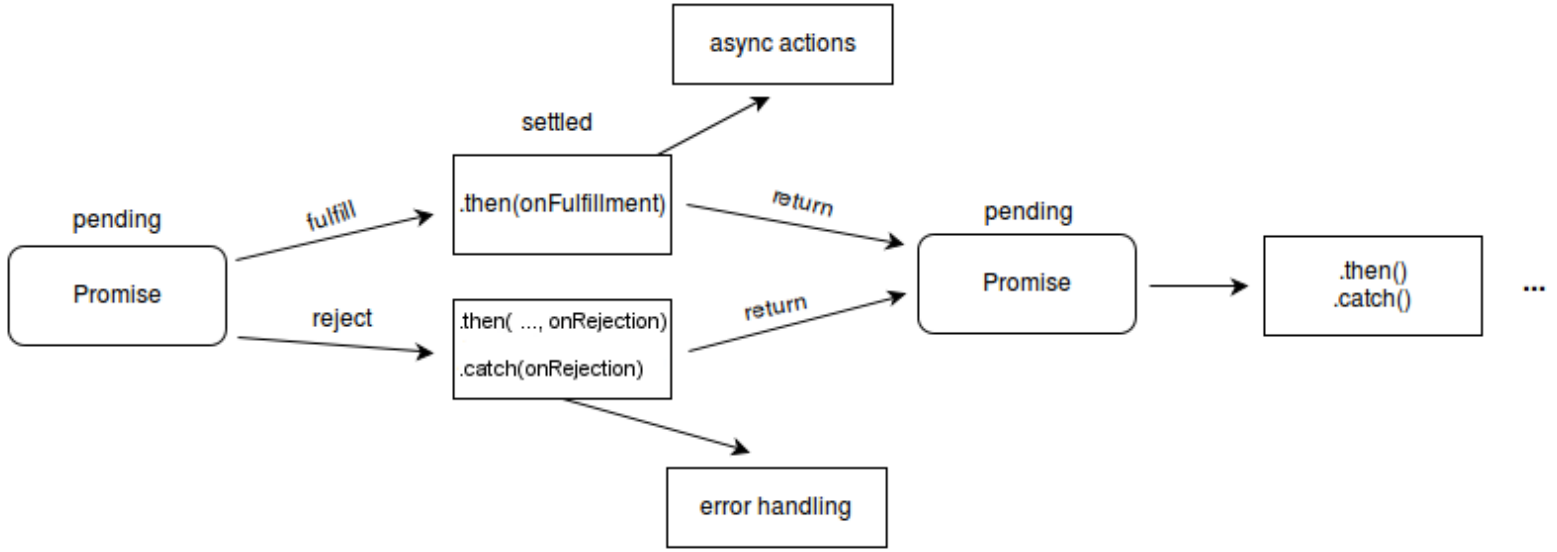
COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES6 (ES2015): Promises (cont.)

- A promise is in one of the following states:
  - *pending:* initial state, neither fulfilled nor rejected.
  - *fulfilled*: meaning that the operation was completed successfully.
  - *rejected*: meaning that the operation failed.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES6 (ES2015): Promises (cont.)



- A pending promise can either be **fulfilled** with a value or **rejected** with a reason (error).
- **`then`** method is called

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES6 (ES2015): Promises (cont.)

- The constructor syntax for a promise object

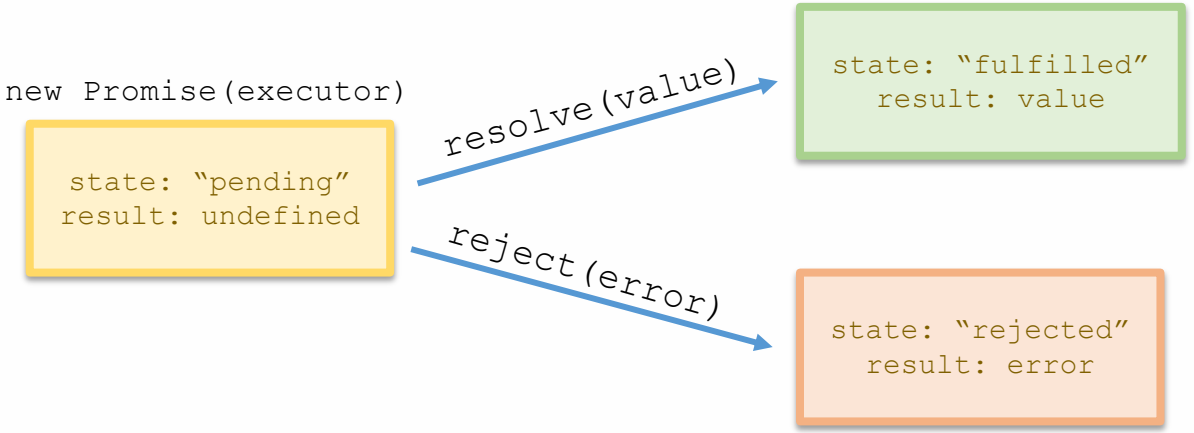
```
let promise = new Promise(function(resolve, reject) {
// executor (the producing code)
});
```

- **`resolve`** and **`reject`** are callbacks. The code is only inside the executor
  - `resolve(value)` — if the job finished successfully, with result value.
  - `reject(error)` — if an error occurred, error is the error object.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES6 (ES2015): Promises (cont.)

- Let observe the internal properties of promise object
  - **state**: initially “pending”, then changes to either “fulfilled” when resolve is called or “rejected” when reject is called.
  - **result**: initially *undefined*, then changes to *value* when `resolve(value)` called or *error* when `reject(error)` is called.



COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES6 (ES2015): Promises (cont.)

- Let see an example (producing code)

```
let myPromise = new Promise(function (resolve, reject) {
let x = 5;

if (x == 0) {
// after 1 second signal that the job is done with the
result "finished"
setTimeout(() => resolve("finished"), 1000);
} else {
// after 1 second signal that the job is finished with an
error
setTimeout(() => reject(new Error("No No!")), 1000);
}
});
```

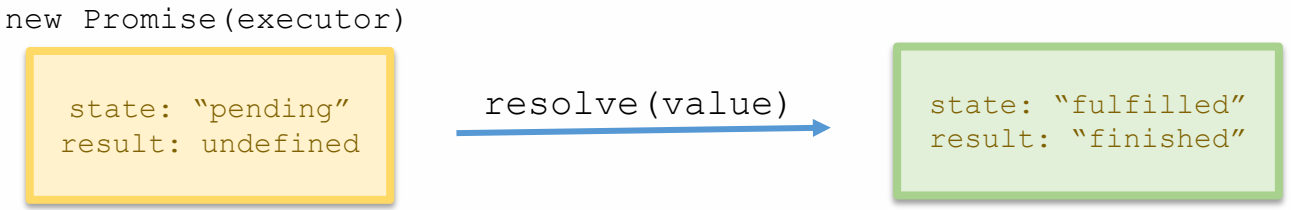
- These functions `resolve` and `reject` are pre-defined by the JavaScript engine

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES6 (ES2015): Promises (cont.)


```
let myPromise = new Promise(function (resolve, reject) {
let x = 5;

if (x == 0) {
// after 1 second signal that the job is done with the
result "finished"
setTimeout(() => resolve("finished"), 1000);
} else {
// after 1 second signal that the job is finished with an
error
setTimeout(() => reject(new Error("No No!")), 1000);
}
});
```

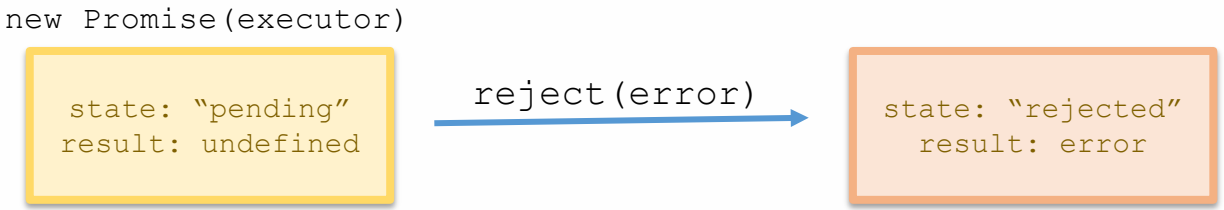


COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES6 (ES2015): Promises (cont.)


```
let myPromise = new Promise(function (resolve, reject) {
let x = 5;

if (x == 0) {
// after 1 second signal that the job is done with the
result "finished"
setTimeout(() => resolve("finished"), 1000);
} else {
// after 1 second signal that the job is finished with an
error
setTimeout(() => reject(new Error("No No!")), 1000);
}
});
```



COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES6 (ES2015): Promises (cont.)

- Now, we will look at consumers.
- Consuming functions can be registered (subscribed) using methods `.then`, `.catch` and `.finally`.
- `then`

```
promise.then(
        function(result) { /* handle a successful result */ },
        function(error) { /* handle an error */
} );
```

  - The first argument of `.then` is a function that executes when the promise is resolved and receives the result.
  - The second argument of `.then` is a function that executes when the promise is rejected and receives the error.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES6 (ES2015): Promises (cont.)

- `then` example

```
myPromise.then(
        function (value) {
                Display(value);
        },
        function (error) {
                Display(error);
        }
);
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES6 (ES2015): Promises (cont.)

- In the case that you need to emphasize on an error, you can use `.catch`

```
myPromise.then(
        function (value) {
                Display(value);
        }
)
.catch((error) => alert(error));
```

- Or in some cases, you are interested only in errors.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES6 (ES2015): Template Literals

- Template Literals is a way to create strings (using backtick “`”)

```
const aString = `A String`;
```

- A way to embed expressions into strings using

```
${a_variable}
```

- Examples,

```
const name = "Wudhichart";
const string = `Hello, ${name}`;
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES6 (ES2015): Template Literals (cont.)

- Tagged templates are used to transfrom the literal string into the formatted string.
- Usage:

```
let aStr = tag`Hello`
```

- `tag` is a tag template that can be any function.

```
function tag(literals, ...substitutions) {
// return a string
}
```

- The `literals` is an array of literal strings.
- The `substitutions` is an array of expressions.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES6 (ES2015): Default parameters

- Functions support default parameters

```
const someFunction = function(index = 0, value = true) {
/* ... */ }
someFunction()
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES6 (ES2015): The spread operator

- An array, object, or string can be expanded using the spread operator (. . .)
- Examples,

```
const a = [1, 2, 3]; //a = 1,2,3
const b = [...a, 4, 5, 6]; //b = 1,2,3,4,5,6
```

- You can make a copy of an array using the spread operator.

```
const a = [1, 2, 3]; //a = 1,2,3
const c = [...a]; //c = 1,2,3
```

- Note that this can be used for an object and a string.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES6 (ES2015): The spread operator

- Converting a string to an array

```
const message = "Hello"; //message = Hello
const arrayized = [...message]; //arrayized = H,e,l,l,o
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES6 (ES2015): Destructuring assignments

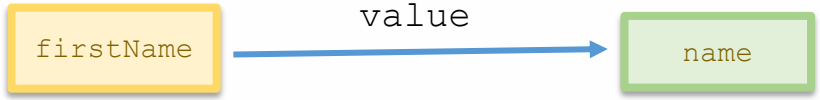
- Some values of object and array can be extracted
- Example of an array

```
let arr = [1, 2];
let [one, two] = arr; //one = 1, two = 2
```

- Example of an object

```
const person = {
firstName: "Wudhichart",
lastName: "Sawangphol",
lecturer: true,
age: 30,
};

let { firstName: name, age } = person;
//name = Wudhichart, age = 30
```



COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES6 (ES2015): For-of loop

- In ES6, `for-of` loop has been introduced.
- This loop can used to iterates over iterable objects, e.g., array
- Examples,

```
for (const ele of ["a", "b", "c"]) {
console.log(ele);
}
```

- Unlike `forEach()`, you can use `break`, `continue`, and `return`

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES6 (ES2015): Map and Set

- Map and Set are introduced.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES6 (ES2015): Map and Set (cont.)

- WeakMap was introduced in ES6.
- WeakMap can only has a key as an object.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES6 (ES2015): Map and Set (cont.)

- The differences between Map and WeakMap

Metrics	Map	WeakMap
Key Types	Any value	Objects
Garbage Collection	No this features	Can be garbage collected if there are no other references to them
Iteration	Iterated over using methods like `keys()`, `values()`, and `entries()`	Do not have these methods because they are not enumerable
Size	Maps have a `size` property	WeakMaps do not have a `size` property
Performance	Fast	Slow

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES6 (ES2015): Map and Set (cont.)

- When to use Maps
  - Use any data type as a key
  - Iterate over the keys and values
  - Get the number of key-value pairs in the map
- When to use WeakMaps
  - Associate data with an object that may be destroyed, like a DOM node
  - Avoid memory leaks

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES6 (ES2015): New String and Object methods

- Some new string and object methods are introduced
  - `repeat()` repeats the strings with a specific number, e.g.,

```
'Hello'.repeat(3) //HelloHelloHello
```

  - `object.is()` determines if two values are the same value
  - `Array.find()` returns the first value of an array that passes a test function

```
let numbers = [10, 30, 25, 48];
let answer = numbers.find((num) => num < 30);
// Answer = 10
```

  - `Array.findIndex()` returns the index of first value of an array that passes a test function

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES6 (ES2015): Function Rest Parameter

- The **rest parameter** (. . .) allows us to have indefinite number of arguments.
- The symbol is the same as the spread operator.
- Example,

```
function findMax(...args) {
      let max = 0;
      for (let arg of args) {
            max = max < arg ? arg : max;
      }
      return max;
}
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.



ES2016

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

- `Array.includes()`
- Exponentiation Operator

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES2016: `Array.includes()`

- With ES6, checking whether an element in an array, we can use `indexOf`.
  - If the value returned is -1, the array does not contain the element that you are looking for.
  - Otherwise, the element is in the array
- With ES2016, checking whether an element in an array, we can use includes.
  - This function returns boolean value.

```
let numbers = [10, 30, 25, 48];
let found = numbers.includes(30); //Found = true
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.





ES2017

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES2017

- String padding
- `Object.values()`
- `Object.entries()`
- `Object.getOwnPropertyDescriptors()`
- Trailing commas
- Async functions

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES2017: String padding

- There are two new String methods: `padStart` and `padEnd` added in ES2017

```
padStart(targetLength [, padString])
padEnd(targetLength [, padString])
```

- Examples,

```
let string = "World";
console.log(string.padStart(10)); // "     World"
console.log(string.padStart(10, "Hello")); // "HelloWorld"
console.log(string.padEnd(10)); // "World.     "
console.log(string.padEnd(10, "Hello")); // "WorldHello"
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES2017: `Object.values()` and `Object.entries()`

- `Object.values()` returns an array of the values of properties of an object

```
const person = {
firstName: "Wudhichart",
lastName: "Sawangphol",
lecturer: true,
age: 30,
};
```

```
let vals = Object.values(person);
//["Wudhichart", "Sawangphol", true,
30]
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES2017: `Object.values()` and `Object.entries()` (cont.)

- `Object.entries()` returns an array of key-value pairs of properties of an object

```
const person = {
firstName: "Wudhichart",
lastName: "Sawangphol",
lecturer: true,
age: 30,
};
```

```
let ents = Object.entries(person);
// [Array(2), Array(2), Array(2), Array(2)]
// 0: (2) ["firstName", "Wudhichart"]
// 1: (2) ["lastName", "Sawangphol"]
// 2: (2) ["lecturer", true]
// 3: (2) ["age", 30]
```

- Note that these 2 methods work with arrays

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES2017: Object.getOwnPropertyDescriptors()

- This methods returns all own properties descriptors of an object.

```
const person = {
firstName: "Wudhichart",
lastName: "Sawangphol",
lecturer: true,
age: 30,
};
```

```
const person2 = {
firstName: "Wudhichart",
lastName: "Sawangphol",
lecturer: true,
age: 30,
};
```

```
const person2 = {};
Object.defineProperties(
person2,
Object.getOwnPropertyDescriptors(person)
);
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES2017: Trailing commas

- This feature allows to have trailing commas in function declarations, and in functions calls

```
const person = {
firstName: "Wudhichart",
lastName: "Sawangphol",
lecturer: true,
age: 30,
};
```

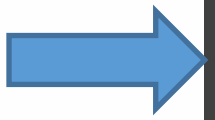
COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES2017: Async functions

- Async/await is a special syntax that works with promises.
- The word “async” before a function means that a function always returns a promise.

```
let myPromise = new Promise(function (resolve, reject)
{
let x = 0;

if (x == 0) {
// after 1 second signal that the job is done with the
result "finished"
setTimeout(() => resolve("finished"), 1000);
} else {
// after 1 second signal that the job is finished with
an error
setTimeout(() => reject(new Error("No No!")), 1000);
}
});
```



```
function resolveAfter1second() {
return myPromise;
}

const asyncFunction = async () => {
console.log(await resolveAfter1second());
};

console.log("Before");
asyncFunction();
console.log("After");
```

```
//Before
//After
//finished
```

- The word “await” makes JavaScript wait until that promise settles and returns its result.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES2017: Async functions (cont.)

- More examples,

```
async function f() {
return 2;
}
f().then(alert);
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.





ES2018

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

- Rest/Spread Properties
- `for-await-of`

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES2018: Rest/Spread Properties

- Rest properties

```
const { firstName, lastName, ...otherProperties } = {
firstName: "Wudhichart",
lastName: "Sawangphol",
lecturer: true,
age: 30,
};
```

- Spread properties

```
const person2 = { firstName, lastName, ...otherProperties };
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES2018: for-await-of

- The new syntax **`for-await-of`** allows you to use async iterable object as the loop iteration
- Example,

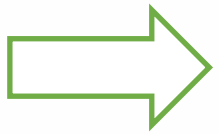
```
const a = [1, 2, 3];

const calMultiply = (value, multiplier) =>
new Promise((resolve) =>
resolve(`${multiplier} * ${value} = ${multiplier * value}`)
);
```

This function is used to multiply value with multiplier.

```
console.log("Start");
const processArray = async (numbers) => {
numbers.forEach(async (element) => {
const val = await calMultiply(element, 2);
console.log(val);
});
console.log("Finished");
};
```

Without for-await-of



```
// Start
// Finished
// 2 * 1 = 2
// 2 * 2 = 4
// 2 * 3 = 6
```

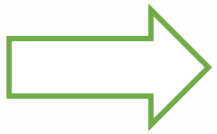
COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES2018: `for-await-of` (cont.)


```
console.log("Start");
const processArrayWithWait = async (numbers) => {
for await (let element of numbers) {
calMultiply(element, 3).then((val) =>
console.log(val));
}
console.log("Finished");
};

processArrayWithWait(a);
```

Without for-await-of



```
// Start
// 3 * 1 = 3
// 3 * 2 = 6
// 3 * 3 = 9
// Finished
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.





ES2019

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

- `Array.prototype.{flat,flatMap}`
- `Object.fromEntries()`
- `String.prototype.{trimStart,trimEnd}`

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

- A multidimensional array can be flated using `.flat` instance method

```
const array = ["Apple", "Orange", ["Grape", ["Pineapple"]]];
```



```
console.log(array.flat()); // ["Apple", "Orange", "Grape", Array(1)]
```



```
console.log(array.flat(2)); // ["Apple", "Orange", "Grape", "Pineapple"]
```

This is a level of multidimensional array

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES2019: `Array.prototype.{flat,flatMap}`

- `.flatmap()` is an Array instance method that combine between `.flat()` and `.map()`

```
const numbers = [1, 2, 3, 4, 5, 6];
```

```
console.log(numbers.map((num) => [num, num * 2]));
// [Array(2), Array(2), Array(2), Array(2),
Array(2), Array(2)]
```

```
console.log(numbers.flatMap((num) => [num, num * 2]));
// [1, 2, 2, 4, 3, 6, 4, 8, 5, 10, 6, 12]
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES2019: Object.fromEntries()

- This method is used to create a new object form properties.

```
const person = {
firstName: "Wudhichart",
lastName: "Sawangphol",
lecturer: true,
age: 30, //made up
};

const entries = Object.entries(person);
const person2 = Object.fromEntries(entries);
// Note that person and person2 are not equivalent
```

This is a new object.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

- `trimStart` and `trimEnd` are used to remove white space from the **start** and **end** of the original string respectively

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.



ES2020

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

- BigInt
- Nullish coalescing operator
- Optional chaining operator
- String.matchAll()

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES2020: BigInt

- BigInt is a new primitive data type
- There are no limits - except for your device's memory.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES2020: Nullish coalescing operator

- A new JS operator, **Nullish coalescing operator** (??), is introduced.
- This operator provides a default value when the variable is either **null** or **undefined**.

```
const name = "Wudhichart";
const nullish = null;

const name2 = name ?? "Jidapa"; //Wudhichart
const name3 = nullish ?? "Jidapa"; //Jidapa
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES2020: Optional chaining operator

- The **optional chaining operator** (? .) handles **null** and **undefined** in objects.

```
const obj = {
prop: {
subLevelProp: {
value: "Wudhichart",
},
},
};

console.log(obj.prop.subLevelProp.value); // Wudhichart
console.log(obj.prop.secondsubLevelProp.value); // error

console.log(obj?.prop?.subLevelProp?.value); // Wudhichart
console.log(obj?.prop?.secondsubLevelProp?.value); // undefined
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES2020: Optional chaining operator (cont.)

- This operator can be used for an array as well.

```
const cars = null;
console.log(cars[3]); // TypeError: Cannot read property '3' of null
console.log(cars?.[3]); // undefined
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES2020: String.matchAll()

- String.matchAll() is used to apply the same regular expression to a string and find all matches.

```
const regexg = /\b(orange)+\b/g;
const fruits = "orange, banana, orange,
apple, orange, apple";
const matched = fruits.matchAll(regexg);
for (result of matched) {
console.log(result);
}
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.





ES2021

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

- Numeric Separator
- String.prototype.replaceAll()
- Logical Assignment Operators
- WeakRef and Finalizers

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES2021: Numeric Separator

- The underscore can be used as Numberic Separator.

```
const billion = 1000_000_000;
console.log(billion);
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES2021: String.prototype.replaceAll()

- `String.prototype.replaceAll()` replaces all occurrence of a string with another string value.

```
const str =
"Wudhichart waits for the man on the stage at the first row as the
first person.";
const newStr = str.replaceAll("first", "last");
console.log(newStr);
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES2021: Logical Assignment Operators

- The Logical assignment operator combines the logical operations(`&&`, `||` or `??`) with the assignment.
  - The logical AND assignment (&&=) operator only evaluates the right operand and assigns it to the left if the left operand is truthy.
  - The logical OR assignment (||=) operator only evaluates the right operand and assigns it to the left if the left operand is falsy.
  - The nullish coalescing assignment (??=) operator only evaluates the right operand and assigns to the left if the left operand is nullish (null or undefined).

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES2021: Logical Assignment Operators (cont.)

- Logical AND assignment (&&=) operator

```
let x = 1;
let y = 0;
let z = 5;

x &&= z;
console.log(x);
// Expected output: 5

y &&= z;
console.log(y);
// Expected output: 0
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES2021: Logical Assignment Operators (cont.)

- Logical OR assignment (||=) operator

```
y = 0;
x = 1;
z = 5;
x ||= z;
console.log(x);
// Expected output: 1

y ||= z;
console.log(y);
// Expected output: 5
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES2021: Logical Assignment Operators (cont.)

- Nullish coalescing assignment (??=) operator

```
const a = { age: 36 };

a.age ??= 20;
console.log(a.age);
// Expected output: 36

a.height ??= 175;
console.log(a.height);
// Expected output: 175
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.





ES2022

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

- Method `.at()` of indexable values.
- `Object.hasOwn(obj, propKey)`
- Cause of `error`
- Top-level await modules – JavaScript allows you to use await outside function.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES2022: Method `.at()` of indexable values.


```
const arr = [1,2,3,4]
console.log(arr.at(2)) //3

const str = "1234"
console.log(str.at(3)) //4
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES2022: `Object.hasOwn(obj, propKey)`


```
let object = { foo: false }
console.log(Object.hasOwn(object, "foo")) // true
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES2022: Cause of `error`


```
try {
doSomeComputationThatThrowAnError()
} catch (error) {
throw new Error('This is error', { cause: error })
}
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.





ES2023

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

- Find array from the last
- Symbols as WeakMap keys
- Change Array by Copy

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES2023: Find array from the last

- This function allows us to find element from the last to first of the array based on a condition.
- There are 2 new functions:
  - `findLast()`
  - `findLastIndex()`
- Example:

```
const aArray = [5, 10, 15, 20];
const aLastElement = aArray.findLast((n) => n > 10);
const aLastIndex = aArray.findLastIndex((n) => n > 10);
console.log(`The last element is ${aLastElement} at
${aLastIndex}.`);
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES2023: Symbols as WeakMap keys

- In ES2023, we can use Symbols as keys in WeakMap.

```
const weakMap = new WeakMap();
const key = Symbol("my symbol");
const someValue = 5;
weakMap.set(key, someValue);
console.log(weakMap.get(key));
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES2023: Change Array by Copy

- There are some additional array methods that make change to an array by returning a new copy.
  - `Array.prototype.toReversed()`
  - `Array.prototype.toSorted(compareFn)`
  - `Array.prototype.toSpliced(start, deleteCount, ...items)`
  - `Array.prototype.with(index, value)`

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.



ES2024

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

- Well-formed Unicode Strings
- Atomic waitSync
- Top-level await
- The Pipeline Operator
- Records and Tuples
- Decorators
- Temporal API

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Well-formed Unicode Strings

- Managing Unicode strings in JavaScript has frequently presented inconsistencies, especially regarding characters from various languages and symbols.
- Incorrectly formatted Unicode strings can result in unforeseen problems, particularly in applications targeting global users or those that heavily depend on API interactions.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Well-formed Unicode Strings (cont.)

- Example

```
const problematicString = "test \uD800";

console.log(problematicString);
console.log(problematicString.toWellFormed());
```

- The `toWellFormed()` function ensures reliable handling of Unicode for applications processing user input from diverse locales or complex character sets, minimizing encoding errors and enhancing cross-platform compatibility.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Atomic `waitSync` for Synchronous Data Access in Shared Memory

- JavaScript's single-threaded design complicates concurrency management, particularly when sharing memory among threads in worker settings. Maintaining synchronized data access to avoid race conditions can be quite challenging.
- ES2024 presents a technique using `waitSync` that enables threads to pause until a certain condition in shared memory is fulfilled. This guarantees synchronization and preserves data integrity.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Atomic `waitSync` for Synchronous Data Access in Shared Memory

- Example

```
const sharedArray = new Int32Array(new SharedArrayBuffer(1024));

// Example usage in a hypothetical shared memory scenario
function performSynchronizedOperation(index, value) {
        Atomics.waitSync(sharedArray, index, 0); // Wait until condition is met
        sharedArray[index] = value;
        Atomics.notify(sharedArray, index, 1); // Notify other threads of the update
}
```

- `waitSync` is a game-changer for applications that depend on real-time data, including collaborative tools or games. It provides more reliable concurrency control, improving the performance and stability of multi-threaded applications.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Top-level `await`

- In the past, the `await` keyword was restricted to `async` functions, forcing developers to encapsulate code within extra function calls, which resulted in an unwanted boilerplate.
- In ES2024, top-level `await` enables developers to utilize `await` directly at the module level, making asynchronous code more efficient and removing the necessity for additional `async` functions.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Top-level `await` (cont.)

- Example:

```
const data = await fetchData();
console.log(data);
```

- This feature enhances the accessibility and intuitiveness of asynchronous programming.
- Developers who use data-fetching modules or applications that depend on API calls find top-level await to be a valuable improvement, as it streamlines code organization and enhances clarity.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

The Pipeline Operator (|>)

- As the Pipeline Operator is an experimental feature in the stage 1 proposal, it does not support currently available browsers.
- JavaScript developers often rely on nested function calls for data transformations, which can result in convoluted code that is difficult to read and maintain.
- The pipeline operator (|>) allows for functional syntax, passing the output of one function as input to the next. This improves code readability and helps organize complex transformations.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

The Pipeline Operator (|>) (cont.)

- Example

```
const processedValue = -10
      |> (n => Math.max(0, n))
      |> (n => Math.pow(n, 2))
      |> Math.ceil;
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Records and Tuples: Immutability Made Easy

- These data structures are not available in official JavaScript because they are merely proposals for additions to JavaScript, which does not support currently available browsers.
- Keeping data the same in JavaScript has required complicated solutions, especially in applications where managing state is important, such as frameworks like React.
- Records and Tuples are data structures that cannot be changed after creation. This means they are stable and help prevent accidental changes in your code. They serve as simpler alternatives to objects and arrays.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Records and Tuples: Immutability Made Easy

- Example

```
const drivingLicence = #{
country: "Thailand",
number: 123456789870,
};

const wallet = #[
drivingLicence,
];
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Decorators: Modifying Classes Made Simple

- Decorators are a stage 2 proposal for JavaScript, meaning they are not yet part of the ECMAScript standard. However, many modern JavaScript frameworks like Angular and libraries such as TypeScript use them widely.
- A decorator is a function that enhances an existing method, enabling modifications to an object's behavior without altering its original code, thereby extending its functionality.
- Decorators are typically used with classes and prefixed with the @ symbol.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Decorators: Modifying Classes Made Simple (cont.)

- Example

```
// A simple decorator
function log(target, key, descriptor) {
console.log(`Logging ${key} function`);
return descriptor;
}

class Example {
@log
greet() {
console.log("Hello, world!");
}
}

const example = new Example();
example.greet(); // Logs "Logging greet function" and "Hello, world!"
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Decorators: Modifying Classes Made Simple (cont.)

- Decorators can be applied to various elements of a class:
  - **Class Decorators**: Applied to the entire class.
  - **Method Decorators**: Applied to a method.
  - **Accessor Decorators**: Applied to getters and setters.
  - **Property Decorators**: Applied to a class property.
  - **Parameter Decorators**: Applied to a method parameter.
- Decorators enable developers to add features like logging, validation, or access control to classes and methods without cluttering the core logic.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Temporal API

- The Date object is often considered tedious and has been a persistent issue in JavaScript.
- This is why libraries such as moment.js and date-fns exist, as developers use them to manipulate the Date object easily.
- The Date object was originally derived from Java, which later abandoned it, but JavaScript retained it for backward compatibility.
- It hasn't been revised since its inception long ago.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Temporal API (cont.)

- The Temporal API is still experimental. It is currently at Stage 3 in the TC39 process, meaning it is close to being finalized but is not yet part of the official ECMAScript standard.
- The Temporal API aims to enhance date and time management for JavaScript developers, addressing existing limitations.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Why Use Temporal API?

- Comprehensive Features
  - Temporal API provides a comprehensive set of features for efficient date and time management, surpassing the capabilities of the traditional Date object.
- Native Time Zone Support
  - With native support for all time zones, Temporal API ensures accurate and reliable date and time calculations across different regions.
- No External Dependencies
  - Temporal API functions independently without the need for external libraries, simplifying the development process and improving performance.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Key Features of Temporal API

- Support for Time Zones
  - The Temporal API provides first-class support for various time zones, ensuring accurate time calculations across regions.
- String Format Parsing
  - It reliably parses strict string formats, making it easier to handle dates and times from various sources.
- Non-Gregorian Calendars
  - The API supports non-Gregorian calendars, allowing developers to work with diverse calendar systems seamlessly.
- Simplified Date Time Computations
  - It simplifies date and time computations through straightforward APIs, enhancing developer productivity and accessibility.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

ES2025

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

- Import Attributes / JSON Modules
- Iterator Helpers
- Set Methods and Relationships
- RegExp Enhancements
- Promise.try()
- Float16 Support

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Import Attributes / JSON Modules

- ES2025 formalizes the ability to import JSON files directly as modules, with explicit attributes so the runtime knows how to handle the file.

```
// Synchronous import
import config from './config.json' with { type: 'json' };
console.log(config.apiUrl);


// Dynamic import
const cfg = await import('./config.json', { with: { type:
'json' } });
console.log(cfg.default.apiUrl);
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Iterator Helpers

- New methods for lazy functional programming on iterators. These do not eagerly create arrays like .map() or .filter() on arrays.
- Useful for pipelines that process data streams efficiently

```
const it = Iterator.from([1, 2, 3, 4, 5])
.filter(n => n % 2 === 0) // keep evens
.map(n => n * n) // square
.take(2); // first two


console.log(it.toArray()); // [4, 16]
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Set Methods and Relationships

- Set gets mathematical set operations built-in.

```
const A = new Set([1, 2, 3]);
const B = new Set([3, 4, 5]);


console.log(A.union(B)); // Set {1, 2, 3, 4, 5}
console.log(A.intersection(B)); // Set {3}
console.log(A.difference(B)); // Set {1, 2}
console.log(A.symmetricDifference(B));// Set {1, 2,
4, 5}


console.log(A.isSubsetOf(B)); // false
console.log(A.isDisjointFrom(B)); // false
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

RegExp Enhancements

- RegExp.escape() : Safely escape strings for use in regex.

```
const userInput = "hello.*+?^${}()|[]\\world";
const safe = RegExp.escape(userInput);
console.log(safe); //
"hello\.\*\+\?\^\$\{\}\(\)\|\[\]\\world"
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

RegExp Enhancements (cont.)

- Inline RegExp Modifiers: Apply flags only to part of a regex.

```
const re = /a(?i:b)c/; // only 'b' is case-insensitive
console.log("aBc".match(re)); // ✅ match
```

- Duplicate Named Capture Groups: Now allowed across alternatives.

```
const re = /(?:(?<num>\d+)|(?<num>[a-z]+))/;
console.log("123".match(re).groups.num); // "123"
console.log("abc".match(re).groups.num); // "abc"
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Promise.try()

- Standard way to wrap functions that might throw synchronously or return a Promise.
- This makes sure that exceptions are properly handled as promise rejections.

```
function mayThrow() {
if (Math.random() > 0.5) throw new Error("Oops");
return 42;
}


Promise.try(() => mayThrow())
.then(val => console.log("Value:", val))
.catch(err => console.error("Caught:", err.message));
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Float16 Support

- A new `Float16Array` typed array, plus methods for handling half-precision (16-bit) floats.
- `Float16Array` requires half the memory of `Float32Array` and a quarter of `Float64Array`.

```
const arr = new Float16Array([0.1, 0.2, 0.3]);
console.log(arr[0]); // approximated float16 value


const dv = new DataView(new ArrayBuffer(2));
dv.setFloat16(0, Math.PI);
console.log(dv.getFloat16(0)); // rounded float16
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.





ES2026

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

- Accurate Math with `Math.sumPrecise()`
- Error detection with `Error.isError()`
- Asynchronous array creation with `Array.fromAsync()`
- Map upsert with `getOrInsert()`
- High-precision JSON with `rawJSON()`
- Base64 and hexadecimal encodings for `Uint8Array`

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Accurate Math: Math.sumPrecise()

- Floating-point arithmetic in JavaScript is notoriously imprecise.
- `Math.sumPrecise()` calculates the exact sum of an iterable of numbers without precision loss, making financial and scientific computations natively reliable without third-party libraries.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Accurate Math: Math.sumPrecise()

- `Math.sumPrecise` is a very recent ECMAScript proposal and is not yet natively supported in all web browsers.
- If you run it natively today, you will likely get a `Math.sumPrecise` is not a function error.
- We need to add a <script> tag that imports a `core-js` polyfill so you can use this feature today.

```
<!-- Polyfill for Math.sumPrecise since it's very new and may not be supported by your browser yet -->
<script src="https://unpkg.com/core-js-bundle/minified.js"></script>
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Accurate Math: Math.sumPrecise() (cont.)


```
// The Old Way (Inaccurate)
const numbers = [1e16, 1, -1e16];
const badSum = numbers.reduce((a, b) => a + b);
// badSum === 0 (The 1 is lost because 1e16 + 1 is rounded back
to 1e16)
console.log("badSum:", badSum);

// ES2026: The New Way (Precise)
// Math.sumPrecise keeps exact precision internally before
rounding the final result.
const exactSum = Math.sumPrecise(numbers);
// exactSum === 1
console.log("exactSum:", exactSum);
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Error detection with Error.isError()

- Checking if a thrown value is an actual Error object is difficult. `instanceof` Error fails across different Realms (like iframes or Web Workers).
- `Error.isError(value)` provides a secure, built-in mechanism to reliably determine if an unknown value is a genuine Error object, regardless of its origin.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Error detection with Error.isError() (cont.)


```
// 1. We create an iframe (a different "Realm")
const iframe = document.createElement('iframe');
iframe.style.display = 'none';
document.body.appendChild(iframe);
// 2. We get the Error constructor from the iframe's realm
const IframeError = iframe.contentWindow.Error;
try {
// 3. We simulate a function throwing an error from that iframe
throw new IframeError('Something went wrong in the iframe!');
} catch (err) {
// The Old Way: Fails because the error's prototype chain points to the iframe's Error, not our
window's Error.
console.log('err instanceof Error:', err instanceof Error); // false

// ES2026: The New Way: Error.isError() safely checks across all realms
console.log('Error.isError(err):', Error.isError(err)); // true

if (Error.isError(err)) {
console.log('Successfully caught cross-realm error:', err.message);
} else {
throw new Error('Unknown exception');
}
}
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All rights reserved.

Asynchronous array creation with Array.fromAsync()

- Iterating asynchronous streams manually using for await...of loops and pushing to an array is repetitive boilerplate.
- It natively creates a new Array instance from an async iterable, iterable, or array-like object.

```
// Fetching pages of data natively
const data = await Array.fromAsync(
asyncDataStream()
);
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Map upsert with getOrInsert()

- When working with Maps, ensuring a key exists before modifying its value requires verbose conditional logic (checking `.has()`, then `.set()`, then `.get()`).
- `Map.getOrInsert()` retrieves the value for a given key, or computes and inserts a new default value if the key is missing.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Map upsert with getOrInsert() (cont.)


```
// The Old Way (Verbose Boilerplate)
const oldVisits = new Map();
const user1 = 'user_123';

if (!oldVisits.has(user1)) {
oldVisits.set(user1, { count: 0 });
}
const oldStats = oldVisits.get(user1);
oldStats.count++;
console.log('Old Way visits:', oldVisits.get(user1).count); // 1

// ES2026: The New Way (Elegant)
const newVisits = new Map();
const user2 = 'user_456';

// You can use getOrInsert directly by passing the value instead of a callback:
const newStats = newVisits.getOrInsert(
user2,
{ count: 0 }
);
newStats.count++;
console.log('New Way visits:', newVisits.get(user2).count); // 1
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

High-precision JSON with rawJSON()

- Parsing JSON strings containing extremely large numbers (like 64-bit IDs) results in rounding errors because standard JSON.parse() converts them to imprecise floats.
- JSON.rawJSON() enables stringifying and accessing raw JSON text values directly, preventing data mutation on large numbers.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

High-precision JSON with rawJSON() (cont.)


```
// The Old Way (Inaccurate or crashes)
// A number larger than Number.MAX_SAFE_INTEGER loses precision:
const oldPayload = {
name: "System Node",
id: 9007199254740993 // Gets rounded down to 9007199254740992 by JavaScript
};
console.log('Old Way stringified:', JSON.stringify(oldPayload));
// (If you tried using a BigInt like 9007199254740993n, JSON.stringify
throws a TypeError)

// ES2026: The New Way (JSON.rawJSON)
// Allows injecting raw, exact JSON text directly into the output.
const newPayload = {
name: "System Node",
id: JSON.rawJSON("9007199254740993")
};

console.log('New Way stringified:', JSON.stringify(newPayload));
// Output: '{"name":"System Node","id":9007199254740993}'
```

Base64 and hexadecimal encodings for Uint8Array

- Handling binary data encodings previously required using `btoa()` and `atob()` (which only operate on strings, not bytes) or relying on heavy third-party NPM packages.
- Native Base64 and Hexadecimal encoding/decoding methods have been added directly to the `Uint8Array` prototype for seamless binary manipulation.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

Base64 and hexadecimal encodings for Uint8Array (cont.)


```
const bytes = new Uint8Array([72, 101, 108, 108, 111]); // "Hello" in ASCII

// The Old Way (Verbose and often performance heavy)
// btoa() requires converting every byte to a character first.
const oldBase64 = btoa(String.fromCharCode(...bytes)); // Warning: Fails on very
large arrays!
const oldHex = Array.from(bytes).map(b => b.toString(16).padStart(2, '0')).join('');
console.log('Old Way Base64:', oldBase64);
console.log('Old Way Hex:', oldHex);

// ES2026: The New Way (Native, safe, and performant)
const base64Str = bytes.toBase64();
const hexStr = bytes.toHex();
console.log('New Way Base64:', base64Str);
console.log('New Way Hex:', hexStr);

// Native Decoding back to Uint8Array
const decodedFromBase64 = Uint8Array.fromBase64(base64Str);
const decodedFromHex = Uint8Array.fromHex(hexStr);
console.log('Decoded match:', decodedFromBase64[0] === 72);
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

References

- Ecma International. (2026). *ECMAScript® 2027 Language Specification*. https://tc39.es/ecma262
- MDN. (2026). *JavaScript technologies overview*. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/JavaScript_technologies_overview
- MDN. (2026). *JavaScript*. https://developer.mozilla.org/en-US/docs/Web/JavaScript

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.